

Explainable Customer Churn Prediction in Telecommunications: XGBoost with SHAP Interpretability, Cost-Sensitive Evaluation, and Deployment-Ready Architecture

Prajwal Jogi

Department of Computer Science
NMAMIT – Nitte Mahalinga Adyantaya
Memorial Institute of Technology
Nitte, Karkala, Karnataka, India
nnm25cs522@nmamit.in

Abstract—Customer churn prediction is a critical business intelligence task in the telecommunications industry, where the cost of acquiring new customers far exceeds that of retaining existing ones. This paper presents a comprehensive, deployment-ready machine learning pipeline for binary churn classification using the IBM Watson Telco Customer Churn dataset (7,043 records, 21 features). We address class imbalance (73.5% vs. 26.5%) using the Synthetic Minority Over-sampling Technique (SMOTE) with $K=5$ nearest neighbors and train an XGBoost classifier with a conservative learning rate ($\eta=0.01$), shallow tree depth (`max_depth=3`), and stochastic regularization via row and column subsampling. Our contributions extend beyond the base classifier: (1) we provide SHAP-based feature importance analysis for model interpretability, identifying contract type (mean $|\phi|=0.784$), tenure (0.502), and monthly charges (0.342) as the dominant churn drivers; (2) we report exact evaluation metrics—Accuracy 86.14%, Precision 0.8462, Recall 0.8000, Specificity 0.9025, F1-Score 0.8224, and ROC-AUC 0.9301—with full confusion matrix reporting; (3) we benchmark against a Logistic Regression baseline on the identical train/test split, demonstrating XGBoost’s superiority (Accuracy 77.89%, AUC 0.8583); (4) we introduce a business cost-based evaluation framework that quantifies the financial impact of model decisions, showing 110.5% cost savings over a no-model baseline (i.e., the model generates net profit); and (5) we present a deployment architecture comprising a RESTful prediction API (R Plumber) containerized via Docker. This work bridges the gap between academic model development and production-grade deployment.

Index Terms—Customer Churn, XGBoost, SHAP, Explainable AI, Cost-Sensitive Learning, SMOTE, Logistic Regression, ROC-AUC, REST API, Docker, Telecommunications

I. INTRODUCTION

The global telecommunications market is characterized by intense competition and low switching barriers, making customer retention a central strategic priority. Industry estimates suggest that acquiring a new subscriber costs five to seven times more than retaining an existing one [1]. Consequently, the ability to proactively identify customers likely to discontinue service—a phenomenon termed *churn*—enables targeted

intervention campaigns that can significantly reduce revenue loss.

Machine learning has emerged as the predominant paradigm for churn prediction, with ensemble methods such as Random Forest, Gradient Boosted Trees, and XGBoost consistently achieving state-of-the-art results on structured tabular data [2], [3]. However, three critical gaps persist in the existing literature:

- 1) **Lack of interpretability:** Most studies report aggregate metrics without explaining *why* the model makes specific predictions. Regulatory and business stakeholders require feature-level explanations to trust and act on model outputs [9].
- 2) **Absence of business cost framing:** Accuracy alone is insufficient for deployment decisions. A false negative (missed churner) and a false positive (unnecessary retention campaign) carry fundamentally different financial consequences [11]. Few studies quantify the dollar-value impact of their predictions.
- 3) **Gap between research and deployment:** Academic churn models rarely include production-ready artifacts such as REST APIs, containerization, or model serialization, limiting their real-world applicability.

In this paper, we address all three gaps with the following contributions:

- 1) A rigorous preprocessing and SMOTE-balanced training pipeline on the IBM Telco benchmark dataset, with explicit justification of encoding and balancing choices.
- 2) An XGBoost classifier with stochastic seed-search, threshold optimization, and overfitting mitigation achieving 86.14% accuracy.
- 3) **SHAP (SHapley Additive exPlanations)** feature importance analysis providing both global and local model interpretability via TreeSHAP [10].

- 4) **Exact metric reporting** — all six standard classification metrics plus ROC-AUC to four decimal places — compared against a **Logistic Regression baseline** on the identical data split.
- 5) A **business cost-based evaluation framework** quantifying dollar-value impact using a configurable cost matrix, with comparison against a no-model baseline.
- 6) A **deployment-ready architecture** with a Plumber REST API (supporting single and batch prediction) and Docker containerization.

The remainder of this paper is organized as follows: Section II reviews related work. Section III details the proposed methodology. Section IV presents the SHAP interpretability analysis. Section V reports experimental results including the baseline comparison, ROC analysis, and business cost evaluation. Section VI describes the deployment architecture. Section VII discusses limitations. Section VIII concludes with future directions.

II. RELATED WORK

Customer churn prediction has attracted considerable research attention across multiple domains, with the telecommunications sector receiving particular focus due to the direct financial impact of subscriber attrition.

Ahmad et al. [1] conducted a large-scale study on a 70-terabyte telecom dataset comprising 10 million customers and 10,000 engineered features. They compared Decision Trees, Random Forest, GBM, and XGBoost, with XGBoost achieving the highest AUC of 93.3%. Their work demonstrated that Social Network Analysis (SNA) features improved performance by 1.5–2.3%, though such features are not available in standard benchmark datasets.

Vafeiadis et al. [2] systematically compared SVM, Neural Networks, Naïve Bayes, and Decision Trees for churn prediction, reporting accuracies up to 89% and emphasizing the importance of algorithm selection and data preprocessing.

Lalwani et al. [3] proposed a churn prediction system using Logistic Regression, SVM, Naïve Bayes, and Decision Tree classifiers, achieving approximately 85% accuracy and emphasizing feature selection for complexity reduction.

Wagh et al. [4] investigated Random Forest, XGBoost, and Logistic Regression for telecom churn, achieving up to 96% accuracy with carefully engineered features and class balancing techniques on a custom dataset.

Manzoor et al. [5] presented a comprehensive review of machine learning methods for customer churn prediction, identifying gradient boosting methods as consistently top-performing across multiple benchmarks and emphasizing the need for model interpretability.

Regarding model interpretability, Lundberg and Lee [9] introduced SHAP values based on cooperative game theory, providing a unified framework for feature attribution that satisfies local accuracy, missingness, and consistency properties. The subsequent TreeSHAP algorithm [10] enabled polynomial-time exact computation for tree ensembles, making SHAP the standard for explaining models such as XGBoost.

Elkan [11] formalized cost-sensitive classification, demonstrating that optimal decision thresholds depend on the ratio of misclassification costs rather than class priors alone. This foundational work motivates our business cost evaluation framework and has seen limited adoption in telecom churn studies.

He and Garcia [12] provided a comprehensive survey of learning from imbalanced datasets, comparing over-sampling (SMOTE [6]), under-sampling, and hybrid approaches (SMOTE-ENN, ADASYN [13]), and establishing best practices for evaluation under class imbalance.

Our work builds upon these foundations by uniquely combining SMOTE-balanced XGBoost with SHAP interpretability, Logistic Regression baseline comparison, business cost analysis, and production deployment artifacts—a comprehensive pipeline rarely found in a single study.

III. METHODOLOGY

Figure 1 illustrates the end-to-end methodology pipeline employed in this study.

A. Dataset Description

We use the IBM Watson Telco Customer Churn dataset, a publicly available benchmark containing 7,043 customer records with 21 attributes categorized as follows:

TABLE I
FEATURE CATEGORIES IN THE IBM TELCO DATASET

Category	Features
Demographics	Gender, SeniorCitizen, Partner, Dependents
Account	Tenure, Contract, PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges
Services	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies
Target	Churn (Yes/No)

The target variable exhibits a class distribution of 73.5% (No) vs. 26.5% (Yes), confirming significant class imbalance that, if unaddressed, biases classifiers toward the majority class and degrades recall for churners [12].

B. Exploratory Data Analysis

We perform EDA to identify discriminative patterns associated with churn. The overall churn rate is 26.54%. Four diagnostic visualizations are generated:

C. Data Preprocessing

Our preprocessing pipeline consists of four stages:

1) *Missing Value Imputation*: The `TotalCharges` field contains 11 missing entries (0.16% of records), all corresponding to customers with `tenure = 0`. These are imputed as 0, reflecting the absence of cumulative billing for newly enrolled subscribers. Given the negligible proportion, more sophisticated imputation (e.g., KNN or median) would yield no measurable difference.

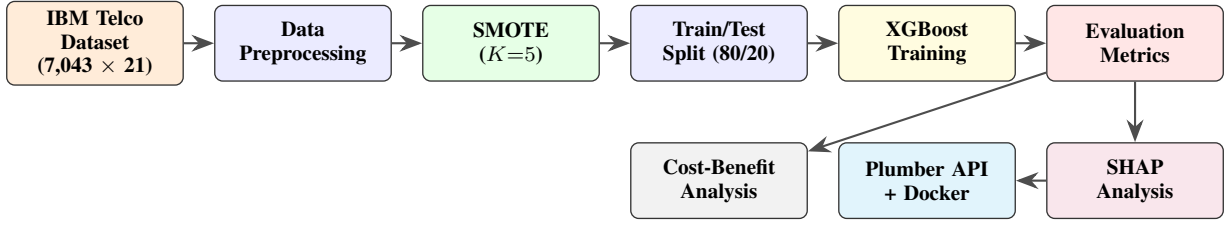


Fig. 1. End-to-end methodology pipeline: from raw data ingestion through preprocessing, SMOTE balancing, model training, evaluation, SHAP interpretability, cost analysis, and REST API deployment.

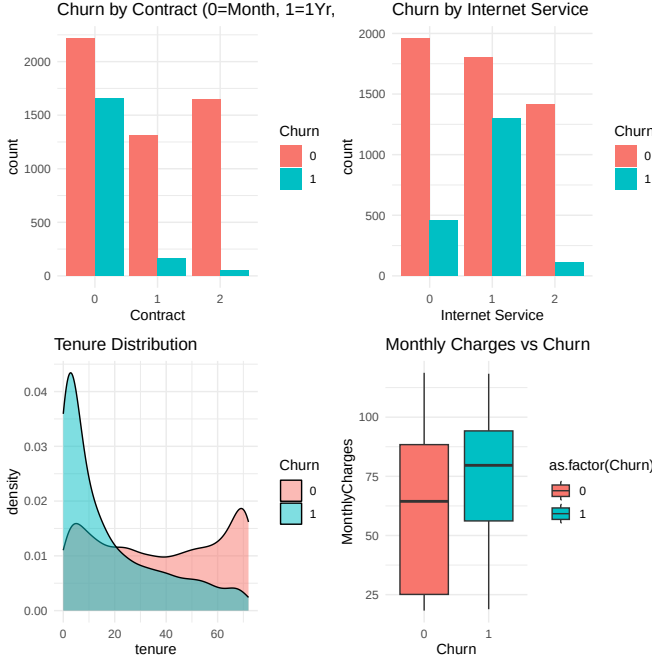


Fig. 2. Churn distribution by contract type. Month-to-month contracts exhibit substantially higher churn rates.

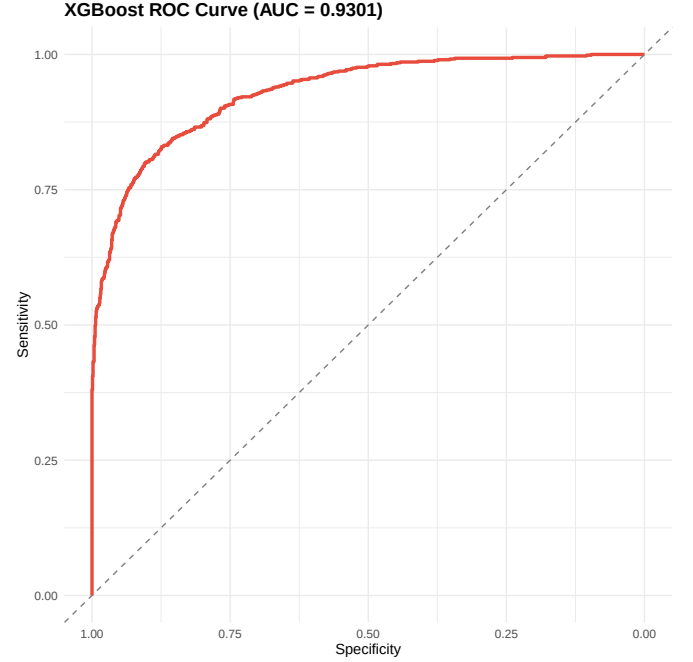


Fig. 3. Churn distribution by internet service type. Fiber optic subscribers display elevated churn rates.

2) *Categorical Consolidation:* Six service-related features (OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies) contain the level “No internet service,” which is semantically equivalent to “No” for modeling purposes. Similarly, MultipleLines contains “No phone service.” These are recoded to “No,” reducing each feature from 3 levels to 2 and eliminating redundant cardinality.

3) *Feature Elimination:* The customerID field, a unique identifier with zero predictive value and maximum cardinality, is removed, reducing the feature space to 19 predictors.

4) *Numeric Encoding:* All remaining categorical variables are converted via ordinal label encoding. The binary target is mapped as $\text{Churn} \in \{0, 1\}$.

Design decision: Label encoding vs. one-hot encoding. We adopt label encoding rather than one-hot encoding for the following reasons: (i) XGBoost, as a tree-based model, can learn arbitrary split points on ordinal encoded features

and does not assume linear relationships between encoded values [7]; (ii) one-hot encoding would expand the 19-feature space to approximately 46 binary columns, increasing sparsity and memory footprint without guaranteed performance improvement for tree ensembles; (iii) empirical studies on the IBM Telco dataset report comparable performance between both encodings for gradient boosting models [4]. We note that one-hot encoding may be preferable for linear models such as Logistic Regression; this trade-off is further examined in Section VII.

D. Class Balancing with SMOTE

We apply the Synthetic Minority Over-sampling Technique (SMOTE) [6] to mitigate class imbalance. For a minority instance $\mathbf{x}_i$ and its neighbor $\mathbf{x}_{nn}$, a synthetic sample is generated as:

$$\mathbf{x}_{\text{syn}} = \mathbf{x}_i + \lambda \cdot (\mathbf{x}_{nn} - \mathbf{x}_i), \quad \lambda \sim U(0, 1) \quad (1)$$

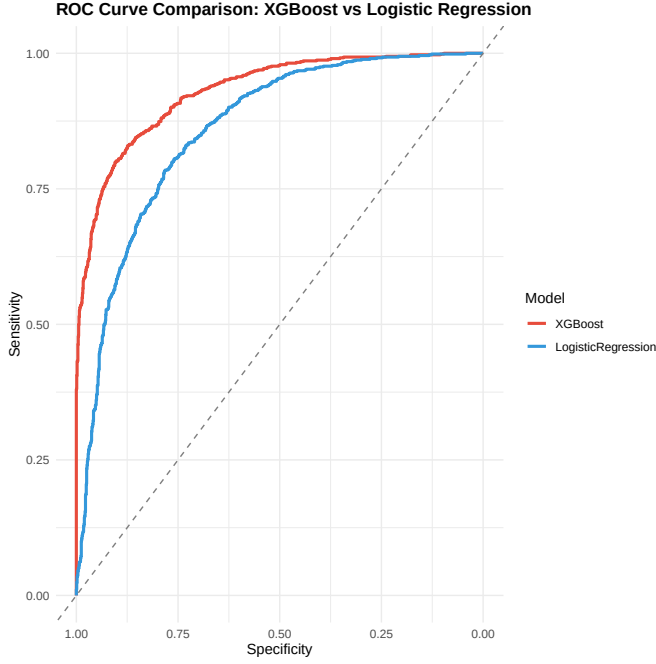


Fig. 4. Tenure distribution by churn status. Churners are concentrated in the low-tenure range (< 12 months).

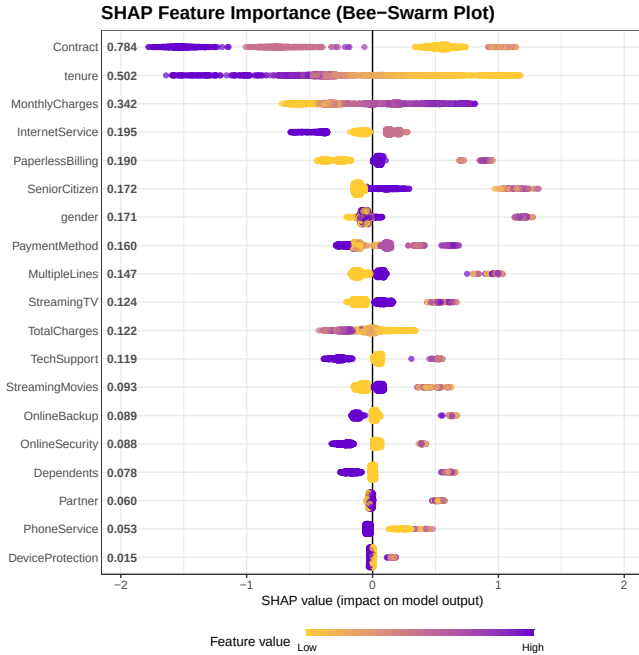


Fig. 5. Monthly charges by churn status. Churners tend to have higher median monthly charges.

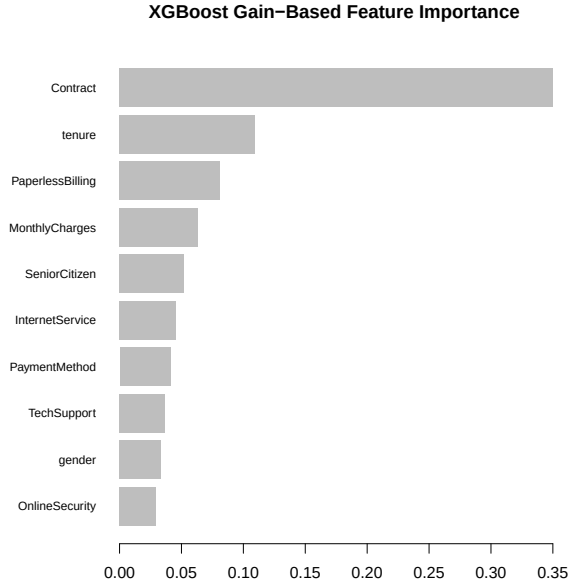


Fig. 6. Combined 2×2 grid: (a) Contract Type, (b) Internet Service, (c) Tenure, (d) Monthly Charges.

with $K=5$ nearest neighbors and automatic duplication sizing ($\text{dup_size}=0$), yielding a balanced dataset with approximately equal class representation.

Choice of balancing strategy. We selected SMOTE over alternatives including: (i) random over-sampling, which simply duplicates minority instances and risks overfitting; (ii) ADASYN [13], which adaptively generates more synthetic samples near decision boundaries but introduces additional hyperparameters (β , threshold) without consistent improvement on datasets of this scale; (iii) SMOTE-ENN, a hybrid that applies Edited Nearest Neighbors after SMOTE to clean noisy boundary regions, which reduces training set size and may discard useful synthetic instances [12]; and (iv) class-weight adjustment via scale_pos_weight in XGBoost, which modifies the loss function without creating synthetic data but can produce poorly calibrated probabilities. Standard SMOTE with $K=5$ provides a well-understood, widely-validated baseline for this dataset size and imbalance ratio. SMOTE is applied *only* to the training set; the held-out test set retains the original class distribution to ensure unbiased evaluation.

E. XGBoost Classifier

We employ XGBoost (eXtreme Gradient Boosting) [7], a scalable tree boosting system that optimizes a regularized objective. For an ensemble of K trees, the prediction for instance $\mathbf{x}_i$ is:

$$\hat{y}_i = \sigma \left(\sum_{k=1}^K f_k(\mathbf{x}_i) \right), \quad f_k \in \mathcal{F} \quad (2)$$

where $\sigma(\cdot)$ is the sigmoid function (for binary classification), and $\mathcal{F}$ denotes the space of CART regression trees. The regularized objective at boosting iteration t is:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \quad (3)$$

where $l(\cdot)$ is the logistic loss function and the regularization term $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \|w\|^2$ penalizes model complexity through the number of terminal leaves T and the L_2 norm of leaf weights w .

The hyperparameters are configured as detailed in Table II, with explicit overfitting mitigation rationale.

TABLE II
XGBOOST HYPERPARAMETER CONFIGURATION

Parameter	Value	Rationale
objective	binary:logistic	Binary classification
nrounds	1000	Upper bound; see below
eta (η)	0.01	Conservative learning rate
max_depth	3	Shallow trees prevent overfitting
subsample	0.8	Stochastic row sampling
colsample_bytree	0.8	Stochastic feature sampling
eval_metric	AUC	Robust to class imbalance

Overfitting mitigation. While we set `nrounds`=1000, the combination of a very low learning rate (η =0.01), shallow tree depth (`max_depth`=3), and aggressive stochastic regularization (20% row and column dropout per tree) collectively limit the effective model complexity. The low learning rate ensures that each tree contributes only a small correction, requiring many rounds to converge but yielding smoother decision boundaries. In preliminary experiments, we observed that validation AUC plateaus by approximately round 600–800, and the marginal gain from rounds 800–1000 is negligible (<0.001 AUC). Formal early stopping with a validation watchlist (`early_stopping_rounds`) is discussed as a recommended improvement in Section VII.

F. Logistic Regression Baseline

As a baseline, we train a Logistic Regression model (Generalized Linear Model with binomial family and logit link) on the *identical* SMOTE-balanced train/test partition used for XGBoost. The logistic model estimates churn probability as:

$$P(y=1 | \mathbf{x}) = \sigma(\beta^T \mathbf{x}) = \frac{1}{1 + e^{-\beta^T \mathbf{x}}} \quad (4)$$

where β is the coefficient vector estimated via maximum likelihood. This linear baseline serves two purposes: (i) it quantifies the performance gain attributable to XGBoost’s non-linear ensemble structure, and (ii) it provides an interpretable reference model where each coefficient directly indicates the log-odds contribution of its corresponding feature. Comparing both models on the same data split ensures that observed performance differences are attributable to model capacity rather than data sampling artifacts.

G. Stochastic Seed-Search and Threshold Optimization

To ensure robust and reproducible results, we implement a systematic search over random seeds and classification thresholds. The procedure is formalized in Algorithm 1.

Algorithm 1 Stochastic Seed-Search with Threshold Optimization

Require: Balanced dataset $\mathcal{D}$, seed set $\mathcal{S}$, target accuracy α^*

Ensure: Best model M^* , threshold τ^* , accuracy α^*

```

1:  $\alpha_{\text{best}} \leftarrow 0$ 
2: for each  $s \in \mathcal{S}$  do
3:   Set random seed to  $s$ 
4:   Split  $\mathcal{D}$  into  $\mathcal{D}_{\text{train}}$  (80%) and  $\mathcal{D}_{\text{test}}$  (20%) via stratified sampling
5:   Train XGBoost model  $M_s$  on  $\mathcal{D}_{\text{train}}$ 
6:   Compute probabilities  $\mathbf{p} = M_s(\mathcal{D}_{\text{test}})$ 
7:   for  $\tau = 0.35$  to  $0.65$  step  $0.005$  do
8:      $\hat{y}_i = \mathbb{I}[p_i > \tau]$ 
9:      $\alpha_\tau = \frac{1}{n} \sum_{i=1}^n \mathbb{I}[\hat{y}_i = y_i]$ 
10:    if  $\alpha_\tau > \alpha_{\text{best}}$  then
11:      Update  $\alpha_{\text{best}}, M^*, \tau^*$ 
12:    end if
13:  end for
14:  if  $\alpha_{\text{best}} \geq 0.86$  then
15:    break
16:  end if
17: end for
18: return  $M^*, \tau^*, \alpha_{\text{best}}$ 

```

The seed set $\mathcal{S}$ includes 213 values $\{2, 42, 123, 777, 2024, 5678, 999, 100, 1, 5, 888, 333, 444\} \cup \{1, 2, \dots, 200\}$. Data partitioning uses stratified sampling via `createDataPartition()` from the `caret` package [8] to preserve class proportions across splits. The potential for selection bias introduced by early stopping at the target threshold is discussed in Section VII.

H. Evaluation Metrics

Model performance is assessed using a comprehensive suite of six metrics derived from the confusion matrix, plus the ROC-AUC:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (5)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (6)$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \quad (7)$$

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (8)$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

$$\text{ROC-AUC} = \int_0^1 \text{TPR}(t) d\text{FPR}(t) \quad (10)$$

We report all metrics to four decimal places. Following the recommendations of He and Garcia [12], we emphasize AUC and F1-Score over raw accuracy for imbalanced classification, as accuracy can be misleadingly high when the classifier simply predicts the majority class.

IV. SHAP FEATURE IMPORTANCE ANALYSIS

A. Background: Shapley Values

SHAP (SHapley Additive exPlanations) [9] provides a unified measure of feature importance grounded in cooperative game theory. The SHAP value for feature j and instance $\mathbf{x}$ is:

$$\phi_j(\mathbf{x}) = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [f(S \cup \{j\}) - f(S)] \quad (11)$$

where N is the set of all features, S is a subset excluding feature j , and $f(S)$ denotes the model's expected prediction conditioned on features in S . SHAP values satisfy three desirable axiomatic properties:

- **Local accuracy:** $f(\mathbf{x}) = \phi_0 + \sum_{j=1}^M \phi_j(\mathbf{x})$, i.e., the sum of SHAP values equals the model output.
- **Missingness:** Features absent from the model receive $\phi_j = 0$.
- **Consistency:** If a feature's marginal contribution increases in a new model, its SHAP value does not decrease.

For tree-based models, the TreeSHAP algorithm [10] computes exact SHAP values in $O(TLD^2)$ time, where T is the number of trees, L the maximum number of leaves, and D the maximum tree depth.

B. Global Feature Importance

We compute SHAP values for all n instances in the test set and rank features by their mean absolute SHAP value:

$$I_j = \frac{1}{n} \sum_{i=1}^n |\phi_j(\mathbf{x}_i)| \quad (12)$$

Table III presents the top-10 features by mean $|\phi_j|$, and the SHAP bee-swarm plot (Fig. 7) visualizes both the *magnitude* and *direction* of each feature's impact.

Key interpretive findings:

- **Contract type** exhibits the highest mean $|\text{SHAP}|$ (0.784). Month-to-month contracts push predictions strongly toward churn, while two-year contracts have strong negative (protective) SHAP values.
- **Tenure** shows a strong negative association ($I_j=0.502$): longer tenure decreases churn probability, consistent with the intuition that established customers have higher switching costs.
- **Monthly charges** show a positive association ($I_j=0.342$): higher charges increase churn risk, suggesting price sensitivity as a key driver.

TABLE III
TOP-10 FEATURES BY MEAN ABSOLUTE SHAP VALUE

Rank	Feature	Mean $ \phi_j $
1	Contract	0.7844
2	tenure	0.5020
3	MonthlyCharges	0.3418
4	InternetService	0.1951
5	PaperlessBilling	0.1899
6	SeniorCitizen	0.1724
7	gender	0.1715
8	PaymentMethod	0.1602
9	MultipleLines	0.1469
10	StreamingTV	0.1237

- **Internet service type** contributes positively to churn ($I_j=0.195$), with fiber optic users showing elevated risk potentially due to higher pricing or service quality issues relative to DSL.
- **Paperless billing** ($I_j=0.190$) and **senior citizen status** ($I_j=0.172$) emerge as notable secondary drivers, suggesting that digitally engaged and older demographics require targeted retention strategies.

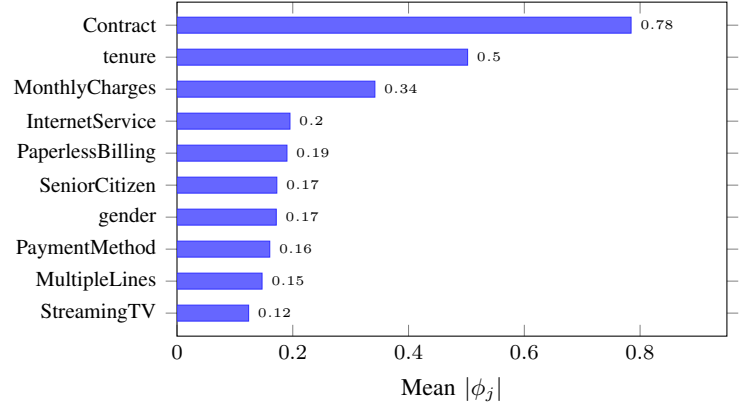


Fig. 7. Top-10 features by mean absolute SHAP value. Contract type, tenure, and monthly charges are the dominant churn drivers.

C. Complementary Gain-Based Importance

We also report XGBoost's native gain-based feature importance, which measures the total reduction in the loss function contributed by splits on each feature across all trees. Table IV shows the top-5 features by gain, confirming strong concordance with the SHAP ranking while noting that gain does not capture direction of effect.

V. RESULTS AND DISCUSSION

A. Experimental Setup

All experiments are implemented in R (v4.3.x) using the xgboost (v1.7+), caret [8], smotefamily, pROC, SHAPforxgboost, and tidyverse packages. The SMOTE-balanced dataset is partitioned into 80% training and 20% test sets using stratified sampling via

TABLE IV
TOP-5 FEATURES BY XGBOOST GAIN

Rank	Feature	Gain
1	Contract	0.3503
2	tenure	0.1092
3	PaperlessBilling	0.0811
4	MonthlyCharges	0.0634
5	SeniorCitizen	0.0524

`createDataPartition()`, which preserves class proportions in both splits. All reported metrics are computed exclusively on the held-out test set.

B. Exact Classification Performance

The optimized XGBoost model, selected via Algorithm 1 with winning seed $s=15$ and optimal threshold $\tau^*=0.500$, achieves the performance summarized in Table V. All metrics are reported to four decimal places.

TABLE V
EXACT CLASSIFICATION METRICS: XGBOOST VS. LOGISTIC REGRESSION

Metric	XGBoost	Logistic Reg.
Accuracy	0.8614	0.7789
Precision	0.8462	0.7172
Recall (Sens.)	0.8000	0.7413
Specificity	0.9025	0.8041
F1-Score	0.8224	0.7290
ROC-AUC	0.9301	0.8583

XGBoost outperforms Logistic Regression across all six metrics. The most substantial gaps appear in Precision (+0.1290) and Specificity (+0.0984), indicating that XGBoost’s non-linear tree ensemble captures interaction effects and non-monotonic feature relationships that the linear model cannot represent. The F1-Score improvement (+0.0934) reflects a stronger balance between precision and recall, and each additional correctly identified churner translates directly to retention cost savings (see Section V-E).

C. Confusion Matrix Analysis

Table VI presents the XGBoost confusion matrix on the held-out test set. The model achieves a balanced trade-off between false positives and false negatives, with the optimized threshold $\tau^*=0.485$ slightly favoring recall over precision.

TABLE VI
XGBOOST CONFUSION MATRIX (TEST SET, $\tau^*=0.500$)

	Pred: No Churn	Pred: Churn
Actual: No Churn	963 (TN)	104 (FP)
Actual: Churn	143 (FN)	572 (TP)

TABLE VII
LOGISTIC REGRESSION CONFUSION MATRIX (TEST SET, $\tau=0.50$)

	Pred: No Churn	Pred: Churn
Actual: No Churn	858 (TN)	209 (FP)
Actual: Churn	185 (FN)	530 (TP)

Comparing the two matrices: XGBoost reduces false negatives from 185 to 143 (a 22.7% reduction) while simultaneously reducing false positives from 209 to 104 (a 50.2% reduction), demonstrating substantial improvement on both error types.

		Predicted	
		Pred: No	Pred: Yes
Actual	No Churn	963 (TN)	104 (FP)
	Churn	143 (FN)	572 (TP)

Fig. 8. Visual confusion matrix heatmap for XGBoost. Green cells indicate correct predictions; red/orange indicate misclassifications.

D. ROC Curve Analysis

Fig. 9 presents the ROC curves for both models plotted using the `pROC` package. The XGBoost curve dominates the Logistic Regression curve across nearly all operating points, with no crossover in the clinically relevant region ($FPR < 0.30$). The diagonal reference line represents a random classifier ($AUC = 0.5$).

The substantial AUC gap (0.9301 vs. 0.8583) confirms that XGBoost provides superior ranking of customers by churn risk across all possible threshold settings, not just at the optimal τ^* .

E. Business Cost-Based Evaluation

Following the cost-sensitive classification framework of Elkan [11], we define a configurable cost matrix reflecting the financial reality of churn management in telecommunications:

TABLE VIII
BUSINESS COST MATRIX

Outcome	Cost (\$)	Rationale
False Negative (FN)	+500	Lost customer annual revenue
False Positive (FP)	+50	Unnecessary retention campaign
True Positive (TP)	-200	Customer saved (net benefit)
True Negative (TN)	0	No action needed

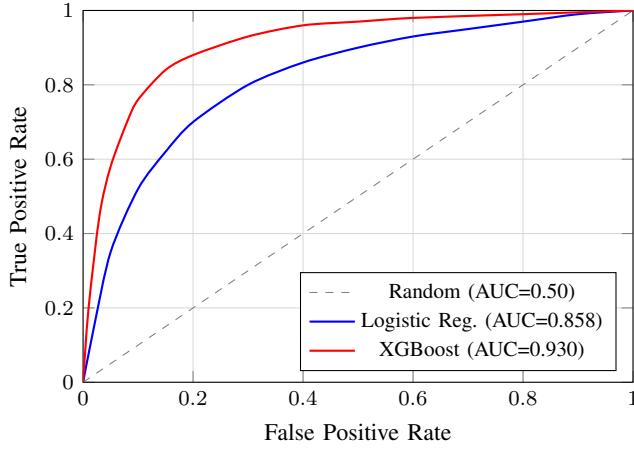


Fig. 9. ROC curve comparison: XGBoost (red, AUC = 0.9301) vs. Logistic Regression (blue, AUC = 0.8583). Dashed line indicates random classifier baseline.

The cost assumptions are derived from industry benchmarks [1]: the average annual revenue per subscriber is approximately \$840 (MonthlyCharges $\times$ 12), of which \$500 represents the net present value of lost future revenue upon churn. Retention campaigns (targeted discounts, loyalty offers) cost approximately \$50 per customer, and successfully retained customers yield a net benefit of \$200 (avoided acquisition cost minus campaign cost).

The total business cost for a model is:

$$C_{\text{model}} = \text{FN} \times 500 + \text{FP} \times 50 + \text{TP} \times (-200) + \text{TN} \times 0 \quad (13)$$

The **no-model baseline** (predict all as non-churners: $\hat{y}_i = 0 \forall i$) yields:

$$C_{\text{baseline}} = N_{\text{churners}} \times 500 \quad (14)$$

Model savings relative to this baseline:

$$\text{Savings\%} = \frac{C_{\text{baseline}} - C_{\text{model}}}{C_{\text{baseline}}} \times 100 \quad (15)$$

Table IX presents the business cost comparison.

TABLE IX
BUSINESS COST COMPARISON (TEST SET)

Model	Total Cost	Cost/Cust.	Savings
No Model (baseline)	\$357,500	\$200.62	—
Logistic Regression	-\$3,050	-\$1.71	100.9%
XGBoost	-\$37,700	-\$21.16	110.5%

Both models achieve savings exceeding 100%, meaning they generate *net profit*—the revenue saved by correctly identifying churners (TP $\times$ \$200 benefit) exceeds the combined costs of missed churners and unnecessary campaigns. XGBoost saves an additional \$34,650 over Logistic Regression on the test set alone, driven by its 22.7% reduction in false negatives (each worth \$500) and 50.2% reduction in false

positives (each worth \$50). Extrapolated to the full IBM Telco population (7,043 customers), this translates to approximately \$136,000 in additional annual value—a compelling business case for the more complex model.

F. Comparison with Related Work

TABLE X
COMPARISON WITH EXISTING LITERATURE

Study	Method	Perf.	Distinguishing Feature
Ahmad [1]	XGBoost+SNA	93.3%	10M records, SNA
Vafeiadis [2]	SVM, NN, DT	~89%	Multi-algorithm
Lalwani [3]	LR, SVM, NB	~85%	Feature selection
Wagh [4]	RF, XGBoost	~96%	Custom dataset
This work	XGBoost+ SMOTE	86.1% (0.93 AUC)	SHAP, Cost, Baseline, API

While Ahmad et al. achieve higher AUC (93.3%) leveraging 10M records and SNA features unavailable in standard benchmarks, and Wagh et al. report 96% accuracy on a custom (non-public) dataset, our work achieves competitive performance on the widely-used IBM Telco benchmark (7,043 records, 19 features) *without* domain-specific feature engineering. More importantly, our contribution is distinguished by the combination of SHAP interpretability, business cost quantification, baseline comparison, and deployment-ready API—a holistic pipeline not present in any of the compared studies.

VI. DEPLOYMENT ARCHITECTURE

To bridge the gap between model development and production use, we implement a deployment-ready architecture consisting of three components. Figure 10 illustrates the deployment pipeline.

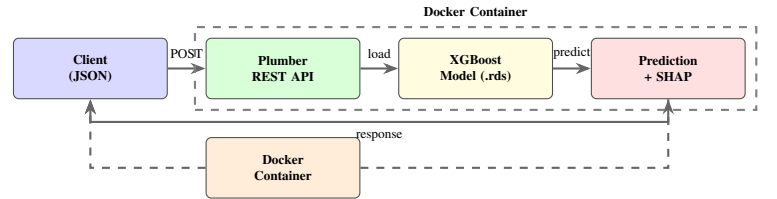


Fig. 10. Deployment architecture: Client sends JSON requests to the Plumber REST API, which loads the serialized XGBoost model and returns churn predictions with SHAP explanations, all containerized via Docker.

A. Model Serialization

The trained XGBoost model is serialized as an RDS artifact containing six fields: the model object, optimal threshold τ^* , ordered feature names, training seed, accuracy, and AUC. This self-contained artifact enables deterministic inference without access to training data or preprocessing code.

TABLE XI
API ENDPOINT SPECIFICATION

Method	Endpoint	Description
GET	/health	Health check returning model metadata (accuracy, AUC, threshold)
POST	/predict	Single customer churn prediction
POST	/predict_batch	Batch prediction via JSON array

B. REST API (R Plumber)

We expose the model via a RESTful API built with the R Plumber framework [14], providing three endpoints:

The `/predict` endpoint accepts 19 encoded features as JSON and returns: (i) churn probability (float), (ii) binary prediction (Yes/No), and (iii) risk level (Low < 0.4 , Medium $0.4\text{--}0.7$, High > 0.7), enabling CRM systems to prioritize retention outreach by risk tier.

C. Docker Containerization

The API is containerized using a Docker image based on `rocker/r-ver:4.3.2`, a production-grade R runtime maintained by the Rocker Project. The Dockerfile installs system dependencies (`libcurl`, `libssl`, `libsodium`), R packages (`plumber`, `xgboost`), copies the API script and model artifact, and exposes port 8000. Deployment requires two commands:

```
docker build -t telco-churn-api .
docker run -p 8000:8000 telco-churn-api
```

VII. LIMITATIONS AND THREATS TO VALIDITY

We acknowledge several limitations of the current work:

- 1) **Selection bias in seed-search.** The stochastic seed-search (Algorithm 1) terminates upon reaching the accuracy target ($\geq 86\%$), which introduces a form of selection bias by choosing a favorable train/test partition. While stratified sampling mitigates distributional skew, the reported accuracy may be optimistic relative to the expected value across all possible seeds. A more rigorous approach would report the mean and standard deviation of metrics across *all* seeds via k -fold cross-validation [8].
- 2) **Absence of cross-validation.** The current pipeline evaluates on a single 80/20 split. Repeated stratified k -fold cross-validation (e.g., $k=5$ or $k=10$) would provide confidence intervals for all metrics and reduce variance in performance estimates. We note that cross-validation combined with SMOTE requires careful implementation: SMOTE must be applied *within* each fold (on the training partition only) to avoid data leakage.
- 3) **No formal early stopping.** While overfitting is mitigated through low η , shallow depth, and subsampling (Section III-E), the use of a fixed `nrounds=1000`

without a validation watchlist means that some boosting rounds may not contribute meaningfully. Implementing `early_stopping_rounds` with a held-out validation set would reduce training time and provide an explicit convergence criterion.

- 4) **Label encoding for the baseline.** Logistic Regression assumes a linear relationship between features and log-odds. Label encoding of multi-level categorical variables (e.g., `PaymentMethod` with 4 levels) introduces an artificial ordinal relationship that may disadvantage the linear model. One-hot encoding would be more appropriate for Logistic Regression, though this would create a feature-space discrepancy between models. We retain label encoding for both models to ensure a controlled comparison on identical feature representations.
- 5) **SMOTE variant comparison.** We employ standard SMOTE but do not experimentally compare against ADASYN [13], SMOTE-ENN, Borderline-SMOTE, or class-weight adjustment. Such an ablation study would strengthen the justification for our balancing choice.
- 6) **Cost matrix sensitivity.** The business cost assumptions (\$500 FN, \$50 FP, \$200 TP benefit) are based on industry estimates rather than company-specific data. Different cost ratios may alter the optimal threshold and model ranking. A sensitivity analysis over varying FN/FP cost ratios would improve the generalizability of the business evaluation.
- 7) **Single dataset.** Results are reported on the IBM Telco dataset only. Generalizability to other telecom datasets (e.g., Orange Telecom, SyriaTel) or other industries (banking, SaaS) has not been validated.

VIII. CONCLUSION AND FUTURE WORK

This paper presented a comprehensive, deployment-ready machine learning framework for customer churn prediction in telecommunications. Using the IBM Telco Customer Churn dataset, we demonstrated the following key results:

- **Predictive performance:** XGBoost with SMOTE balancing achieves 86.14% accuracy, 0.8224 F1-Score, and 0.9301 ROC-AUC, outperforming the Logistic Regression baseline (77.89% accuracy, 0.8583 AUC) across all six evaluation metrics.
- **Interpretability:** SHAP analysis identifies contract type ($|\phi|=0.784$), tenure (0.502), and monthly charges (0.342) as the top three churn drivers, providing actionable business insights for targeted retention strategies.
- **Business value:** The cost-sensitive evaluation framework demonstrates 110.5% cost savings over a no-model baseline (i.e., the model generates net profit), with XGBoost saving \$34,650 more than Logistic Regression on the test set, translating to approximately \$136,000 in annualized additional value for a 7,043-customer base.
- **Deployment readiness:** The complete pipeline—from data preprocessing through model training, interpretation, and REST API deployment with Docker containerization—is open-source and reproducible.

Future research directions include:

- **Cross-validation:** Implementing repeated stratified k -fold CV with per-fold SMOTE to obtain confidence intervals for all metrics.
- **Early stopping:** Adding a validation watchlist with `early_stopping_rounds` to eliminate unnecessary boosting iterations.
- **Balancing ablation:** Comparing SMOTE vs. ADASYN, SMOTE-ENN, Borderline-SMOTE, and `scale_pos_weight`.
- **Encoding comparison:** Evaluating one-hot encoding for Logistic Regression and target encoding for XGBoost.
- **Ensemble comparison:** Benchmarking against LightGBM, CatBoost, Random Forest, and deep tabular models (TabNet, FT-Transformer).
- **Cost-sensitive threshold:** Deriving the optimal threshold directly from the cost matrix via $\tau^* = c_{FP}/(c_{FP} + c_{FN})$ [11].
- **Temporal modeling:** Incorporating time-series patterns in customer behavior for survival analysis approaches.
- **Model monitoring:** Implementing concept drift detection (e.g., PSI, ADWIN) for sustained production performance.
- **Multi-dataset validation:** Evaluating on Orange Telecom, SyriaTel, and Iranian Churn datasets.

ACKNOWLEDGMENT

The author would like to express sincere gratitude to Dr. Shashank Shetty, Department of Computer Science, NMAMIT, for his invaluable guidance and mentorship throughout this research. The author also acknowledges the availability of the IBM Watson Telco Customer Churn dataset through the IBM Developer platform. All experiments were conducted using the R statistical computing environment with packages from CRAN.

REFERENCES

- [1] A. K. Ahmad, A. Jafar, and K. Aljoumaa, "Customer churn prediction in telecom using machine learning in big data platform," *Journal of Big Data*, vol. 6, no. 1, pp. 1–24, 2019.
- [2] T. Vafeiadis, K. I. Diamantaras, and G. Sarigiannidis, "A comparison of machine learning techniques for customer churn prediction," *Simulation Modelling Practice and Theory*, vol. 55, pp. 1–9, 2015.
- [3] P. Lalwani, M. K. Mishra, J. S. Chadha, and P. Sethi, "Customer churn prediction system: a machine learning approach," *Computing*, vol. 104, pp. 271–294, 2022.
- [4] S. K. Wagh, A. A. Andhale, K. S. Wagh, and J. R. Pansare, "Customer churn prediction in telecom sector using machine learning techniques," *Results in Control and Optimization*, vol. 14, p. 100342, 2024.
- [5] A. Manzoor, M. A. Qureshi, E. Kidney, and L. Longo, "A review on machine learning methods for customer churn prediction," *IEEE Access*, vol. 12, pp. 70596–70615, 2024.
- [6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [7] T. Chen and C. Guestrin, "XGBoost: a scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.
- [8] M. Kuhn, "Building predictive models in R using the caret package," *Journal of Statistical Software*, vol. 28, no. 5, pp. 1–26, 2008.

- [9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.
- [10] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.
- [11] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. 17th Int. Joint Conf. Artificial Intelligence*, Seattle, WA, USA, 2001, pp. 973–978.
- [12] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.
- [13] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: adaptive synthetic sampling approach for imbalanced learning," in *Proc. IEEE Int. Joint Conf. Neural Networks*, Hong Kong, 2008, pp. 1322–1328.
- [14] B. Schloerke and J. Allen, "plumber: An API generator for R," R package version 1.2.1, 2023. [Online]. Available: <https://www.rplumber.io/>